

Etude comparative de solveurs pour une plateforme DSL de planification déclarative

Mémoire de Master Recherche en Intelligence Artificielle

6 mai 2026

Résumé

Ce rapport présente une étude expérimentale du choix de solveur pour une plateforme qui traduit des spécifications de planification écrites dans un DSL vers MiniZinc, puis exécute plusieurs solveurs. La contribution principale n'est pas seulement un classement empirique : elle consiste à construire un protocole de benchmark reproductible, à séparer explicitement les instances de satisfaction et les instances d'optimisation, à mesurer plus que la moyenne, puis à interpréter les résultats à la lumière des paradigmes de résolution. La campagne actuelle comporte 10 instances de soutenances, 5 solveurs et 50 exécutions. Tous les solveurs réussissent toutes les instances sans timeout. GECODE obtient la meilleure médiane globale de `T_total` sur ces petites instances, mais CHUFFED obtient la meilleure moyenne globale et le meilleur comportement de passage à l'échelle sur les tailles les plus grandes. La recommandation retenue est donc nuancée : GECODE est très compétitif sur les petites instances simples, tandis que CHUFFED est le candidat le plus robuste pour devenir le solveur par défaut lorsque la taille et la combinatoire augmentent.

Table des matières

1	Introduction	3
2	Contribution méthodologique	3
3	Chaîne de résolution	3
4	SAT ou optimisation dans notre DSL	4
4.1	Instance de satisfaction	4
4.2	Instance d'optimisation	4
5	Instances étudiées	4
6	Métriques expérimentales	5
7	Résultats globaux	5
8	Analyse par famille	8
9	Analyse des statuts et de la robustesse	9
10	Analyse du passage à l'échelle	11
11	Interprétation par la littérature	14
11.1	GEODE : CP classique et petites instances	14
11.2	CHUFFED : lazy clause generation et montée en taille	14
11.3	OR-TOOLS CP-SAT : candidat hybride solide	14
11.4	HIGHS et COIN-BC : solveurs MILP	14
12	Décision expérimentale	15
13	Limites	15
14	Conclusion	16

1 Introduction

Le mémoire porte sur la *conception d'un DSL pour la spécification déclarative de problèmes de planification et leur résolution par des approches de satisfaction de contraintes*. Dans ce cadre, l'utilisateur exprime un problème métier dans un fichier `.planning` : activités, ressources, rôles, contraintes et éventuellement préférences. La plateforme transforme ensuite cette description en MiniZinc, qui sert de passerelle vers plusieurs solveurs.

La question de recherche traitée dans ce rapport est la suivante :

Quel solveur choisir pour résoudre efficacement les problèmes de planification générés par notre DSL, et comment justifier ce choix expérimentalement et théoriquement ?

Le choix du solveur ne peut pas être déduit uniquement du fait que le problème soit un problème de planification. Un même problème peut être formulé comme un problème de satisfaction de contraintes, comme une optimisation en variables entières, comme un problème booléen ou comme un modèle hybride. MiniZinc permet de cibler plusieurs backends, mais cette portabilité ne garantit pas des performances identiques [1]. L'étude doit donc tenir compte de la formulation générée, de la famille d'instance et du comportement empirique des solveurs.

2 Contribution méthodologique

Cette étude apporte trois éléments utiles pour le mémoire.

1. **Une séparation explicite des familles de problèmes.** Une instance de satisfaction contient uniquement des contraintes dures et un tableau `preferences` vide. Une instance d'optimisation contient aussi des préférences, traduites en pénalités à minimiser dans le modèle MiniZinc.
2. **Un générateur d'instances contrôlées.** Le générateur de soutenances produit des instances simples, paramétrées par A , R , K et T , avec des tailles volontairement modestes afin d'éviter que le benchmark mesure seulement des timeouts.
3. **Une analyse statistique multi-critères.** Le pipeline produit les temps de flattening, les temps de résolution, les temps bout en bout, les statuts, les taux de succès, les médianes, les quartiles, les courbes cactus, l'analyse par taille et l'analyse par famille.

Ce protocole transforme le choix d'un solveur en argument scientifique : on ne dit pas seulement qu'un solveur est meilleur, on précise sur quelles familles, avec quelles tailles, selon quelles métriques, et avec quelles limites.

3 Chaîne de résolution

La chaîne évaluée est la suivante :

`.planning` $\longrightarrow$ génération MiniZinc $\longrightarrow$ aplatissage FlatZinc $\longrightarrow$ solveur $\longrightarrow$ solution

Le benchmark est réalisé par `python/benchmarking/benchmark_pipeline.py`. Pour chaque instance, le pipeline génère le modèle MiniZinc via le CLI du projet, exécute les solveurs disponibles avec le même timeout, collecte les statistiques puis produit les artefacts dans `python/benchmarking/result`.

Les solveurs comparés sont :

GECODE, CHUFFED, HIGHS, OR-TOOLS CP-SAT, COIN-BC.

4 SAT ou optimisation dans notre DSL

Dans notre langage, la différence entre une instance de satisfaction et une instance d'optimisation est portée par le bloc `preferences`.

4.1 Instance de satisfaction

Une instance est classée dans la famille **satisfaction** lorsque :

- le bloc `constraints` contient les contraintes obligatoires ;
- le bloc `preferences` est vide ;
- le modèle MiniZinc généré se termine par une résolution de type `satisfy`.

Dans ce cas, le solveur doit seulement trouver un planning valide. Le statut attendu est **SAT** lorsque le planning est trouvé.

4.2 Instance d'optimisation

Une instance est classée dans la famille **optimisation** lorsque :

- le bloc `constraints` contient toujours les contraintes obligatoires ;
- le bloc `preferences` contient au moins une préférence ;
- les préférences sont traduites en une variable de pénalité ;
- le modèle MiniZinc généré se termine par une résolution de type `minimize penalty`.

Dans ce cas, le solveur ne doit pas seulement trouver un planning valide. Il doit aussi minimiser la pénalité. Le statut **OPT** signifie que le solveur a prouvé qu'aucune solution meilleure n'existe selon l'objectif.

Dans notre protocole, la famille du problème vient donc du modèle généré : `preferences = []` implique satisfaction ; `preferences` non vide implique optimisation.

5 Instances étudiées

Les instances de la campagne actuelle sont des instances synthétiques de soutenances générées pour être simples et résolubles rapidement. Elles ne visent pas à saturer les solveurs ; elles servent d'abord à comparer proprement les comportements sur deux familles de problèmes et plusieurs tailles.

Les paramètres sont :

A = nombre de soutenances, R = nombre d'enseignants,
 K = nombre de salles, T = nombre de créneaux par jour.

La campagne contient 10 instances : 5 de satisfaction et 5 d'optimisation. Les deux familles partagent les mêmes tailles afin que la comparaison SAT/OPT soit contrôlée.

TABLE 1 – Instances de la campagne expérimentale

Famille	A	R	K	T	Préférences
satisfaction	3	9	2	3	0
satisfaction	4	12	2	4	0
satisfaction	5	15	3	4	0
satisfaction	6	18	3	5	0
satisfaction	8	24	4	5	0
optimisation	3	9	2	3	3
optimisation	4	12	2	4	3
optimisation	5	15	3	4	3
optimisation	6	18	3	5	3
optimisation	8	24	4	5	3

Les contraintes de base imposent les rôles obligatoires d'une soutenance : président, rapporteur, membre, salle et candidat. Le candidat est fixé pour chaque soutenance. Une partie des jurys est également fixée afin de réduire la symétrie et de produire des instances faciles à résoudre. Les instances d'optimisation ajoutent seulement trois préférences simples ; elles restent volontairement légères pour que le benchmark mesure la qualité de résolution et non une explosion artificielle de complexité.

6 Métriques expérimentales

Les métriques collectées sont :

- **T_flat** : temps de génération/aplatissement du modèle ;
- **T_solve** : temps de recherche du solveur ;
- **T_total** : temps bout en bout observé par le pipeline ;
- **statut** : SAT, OPT, UNSAT, TIMEOUT ou ERROR ;
- **Success rate** : proportion d'exécutions conclues par SAT, OPT ou UNSAT ;
- **OPT rate** : proportion d'exécutions prouvées optimales sur l'ensemble des runs.

Le timeout de référence est 180 secondes par couple solveur-instance. Dans cette campagne, aucun solveur n'atteint ce timeout.

7 Résultats globaux

La campagne contient 50 exécutions, soit 10 instances évaluées par 5 solveurs. Aucun échec, aucun timeout et aucune instance non résolue n'ont été observés. La comparaison se

déplace donc naturellement vers la qualité temporelle de la résolution : temps de recherche pur, temps total bout en bout et sensibilité à la taille.

Le tableau 2 montre immédiatement qu'un classement unique serait réducteur. GECODE possède la meilleure médiane de T_{total} , ce qui traduit un très bon comportement sur le coeur de la distribution. CHUFFED, en revanche, obtient la meilleure moyenne globale ainsi que le meilleur T_{solve} médian. Cette dissociation est importante pour le mémoire : un solveur peut être excellent sur les petites tailles sans être celui qui encaisse le mieux la montée en complexité.

TABLE 2 – Synthèse globale par solveur

Solveur	Moy. T_{total}	Méd. T_{total}	Méd. T_{solve}	Succès	OPT
GECODE	1.024	0.268	0.250	100%	50%
CHUFFED	0.384	0.317	0.062	100%	50%
OR-TOOLS CP-SAT	0.594	0.392	0.263	100%	50%
HIGHS	1.296	1.137	0.546	100%	50%
COIN-BC	1.493	0.888	0.755	100%	50%

La figure 1 met en évidence la dispersion des temps totaux. GECODE garde une médiane très basse, mais sa boîte et sa moustache supérieure montrent qu'il est davantage pénalisé par les grandes instances. CHUFFED présente une distribution plus compacte, signe d'un comportement plus régulier. HIGHS et COIN-BC restent en retrait sur ce benchmark, ce qui suggère que la formulation actuelle favorise davantage la propagation de contraintes et les décisions combinatoires que l'exploitation d'une relaxation linéaire forte.

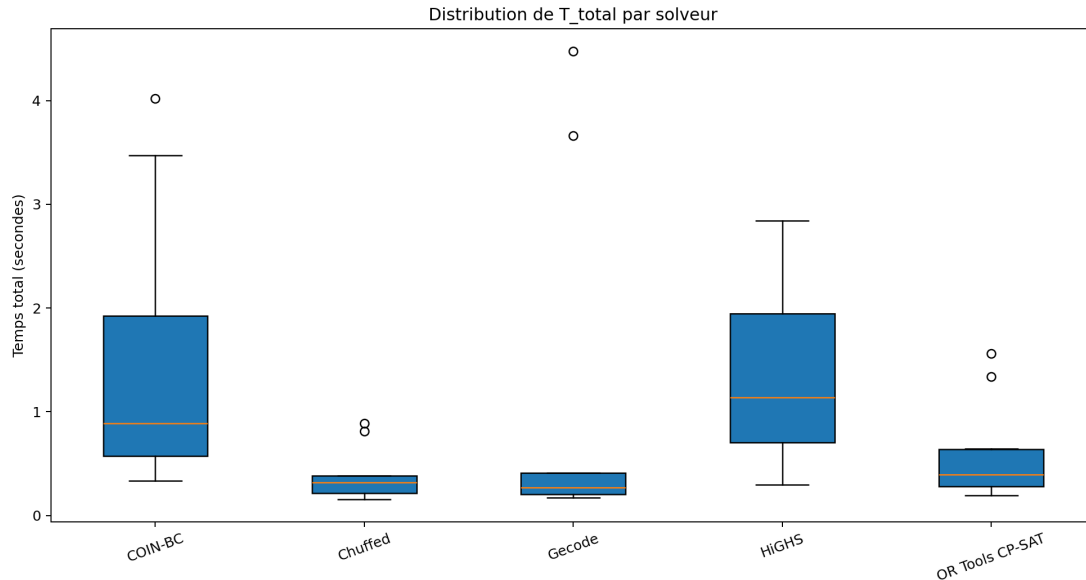


FIGURE 1 – Distribution du temps total T_{total} par solveur sur l'ensemble de la campagne

La figure 2 isole ensuite le temps de recherche. Une fois soustrait le coût d'aplatissement, CHUFFED devient le plus net : sa médiane de T_{solve} est proche de 0.062 seconde,

contre environ 0.250 seconde pour GECODE et 0.263 seconde pour OR-TOOLS CP-SAT. Cette hiérarchie confirme que la partie recherche profite ici de la lazy clause generation, tandis que les solveurs MILP supportent un coût de résolution plus marqué.

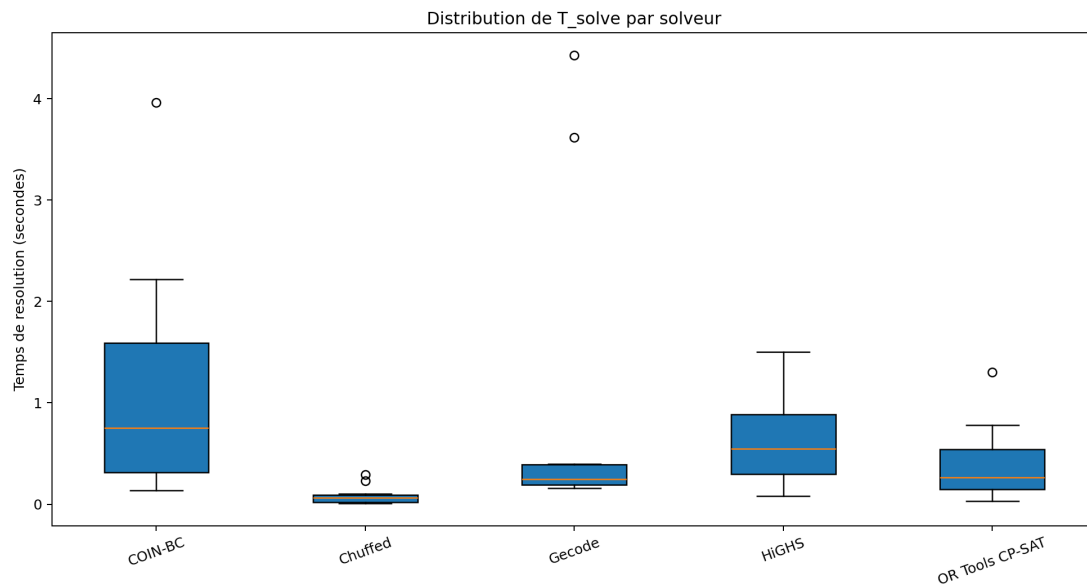


FIGURE 2 – Distribution du temps de recherche T_{solve} par solveur

La figure 3 complète cette lecture en séparant le coût de génération T_{flat} , le coût de recherche T_{solve} et le temps total T_{total} . Sur des instances simples, T_{flat} représente encore une part visible du coût global. Il ne suffit donc pas d'observer le solveur pur. Le solveur retenu pour la plateforme doit offrir un bon équilibre entre traduction MiniZinc, intégration technique et vitesse de recherche. Sur ce point, CHUFFED obtient le compromis le plus stable.

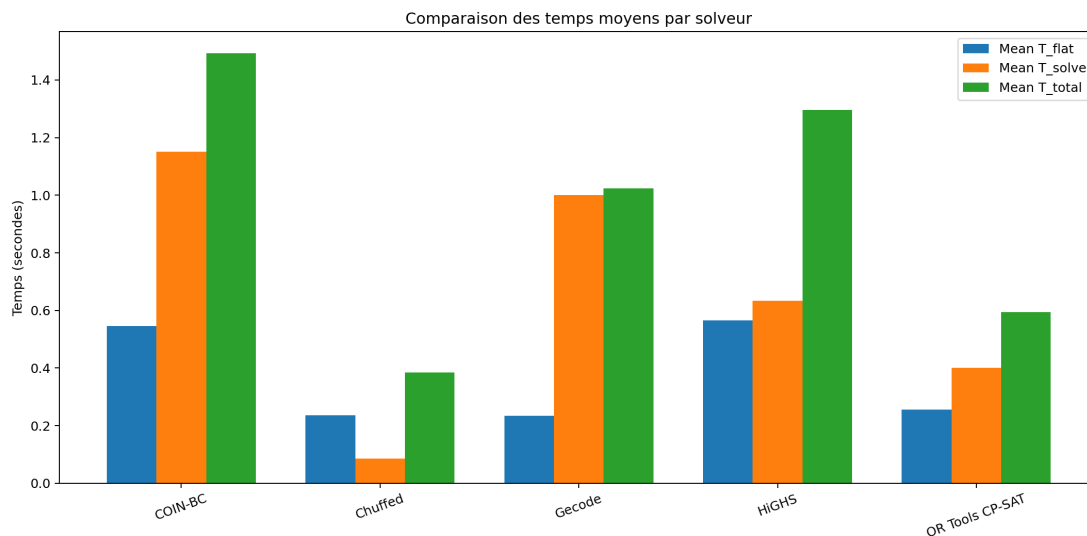


FIGURE 3 – Comparaison des temps moyens de génération, de résolution et de bout en bout

8 Analyse par famille

La séparation entre satisfaction et optimisation est centrale dans notre contribution, car elle évite de mélanger deux comportements de résolution distincts. Dans les instances de satisfaction, le solveur doit uniquement produire un planning valide. Dans les instances d’optimisation, il doit en plus prouver la qualité de la meilleure solution selon l’objectif introduit par les préférences.

Le tableau 3 montre que GECODE garde la meilleure médiane de T_{total} dans les deux familles. L’écart avec CHUFFED reste toutefois modéré, alors que les solveurs MILP demeurent plus lents. Ce résultat indique que les préférences légères introduites dans notre modèle ne suffisent pas encore à déplacer le centre de gravité du problème vers une structure typiquement MILP.

TABLE 3 – Médiane de T_{total} par famille

Famille	Solveur	Runs	Méd. T_{total}	Succès
optimisation	GECODE	5	0.266	100%
optimisation	CHUFFED	5	0.344	100%
optimisation	OR-TOOLS CP-SAT	5	0.392	100%
optimisation	COIN-BC	5	0.913	100%
optimisation	HIGHS	5	1.203	100%
satisfaction	GECODE	5	0.269	100%
satisfaction	CHUFFED	5	0.290	100%
satisfaction	OR-TOOLS CP-SAT	5	0.393	100%
satisfaction	HIGHS	5	0.828	100%
satisfaction	COIN-BC	5	0.862	100%

La figure 4 rend cette comparaison visuelle. Les profils des solveurs restent proches d’une famille à l’autre, ce qui est cohérent avec la façon dont les instances ont été construites : les variantes d’optimisation ne changent pas l’ossature des contraintes, elles ajoutent seulement un petit objectif de pénalité. Le benchmark mesure donc bien l’effet de l’objectif, sans brouiller l’analyse par une modification brutale de la structure du problème.

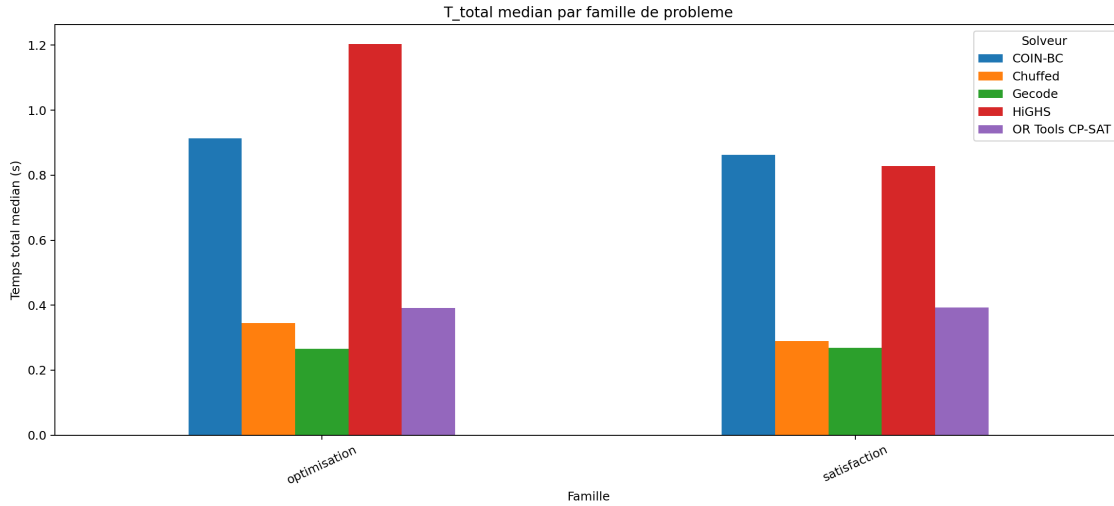


FIGURE 4 – Temps total médian selon la famille du problème

La figure 5 confirme que toutes les instances d’optimisation ont été menées jusqu’à une preuve d’optimalité, quel que soit le solveur. Pour les instances de satisfaction, le taux OPT reste naturellement nul puisqu’aucun objectif n’est demandé. Ce point est méthodologiquement important : notre comparaison ne porte pas sur des solutions partielles ou sur des compromis arrêtés avant preuve, mais bien sur des résultats conclusifs.

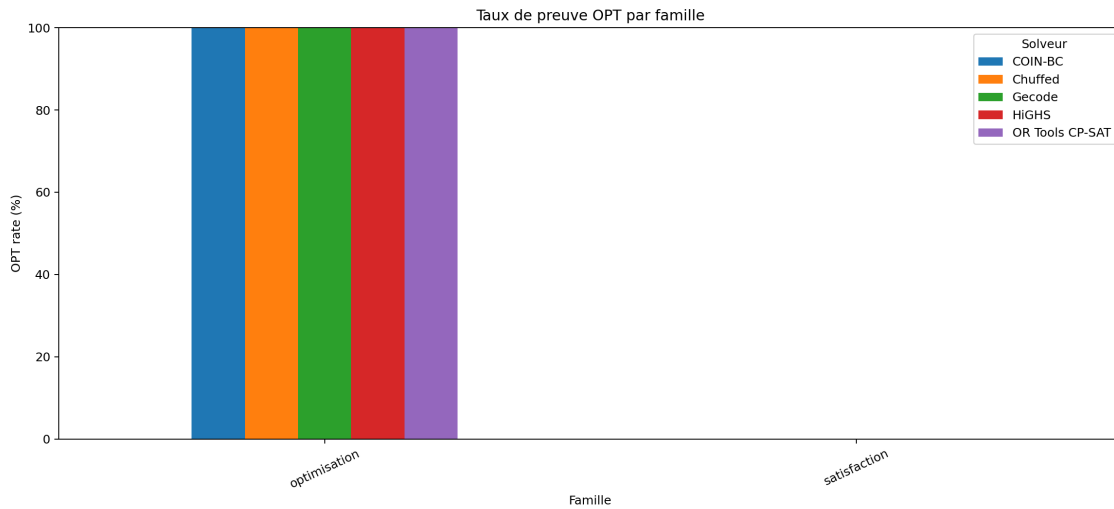


FIGURE 5 – Taux de preuve d’optimalité selon la famille considérée

9 Analyse des statuts et de la robustesse

La robustesse du protocole se vérifie d’abord par les statuts retournés. Le tableau 4 montre une répartition parfaitement conforme à la nature des instances : cinq résolutions OPT pour les instances d’optimisation et cinq résolutions SAT pour les instances de satisfaction, pour chacun des solveurs testés.

TABLE 4 – Répartition des statuts par solveur

Solveur	OPT	SAT
COIN-BC	5	5
CHUFFED	5	5
GECODE	5	5
HIGHS	5	5
OR-TOOLS CP-SAT	5	5

La figure 6 traduit visuellement ce constat. Aucun solveur ne produit d’erreur, de timeout ou de statut ambigu. La campagne est donc suffisamment simple pour isoler le comportement temporel sans être parasitée par des échecs techniques ou des cas pathologiques.

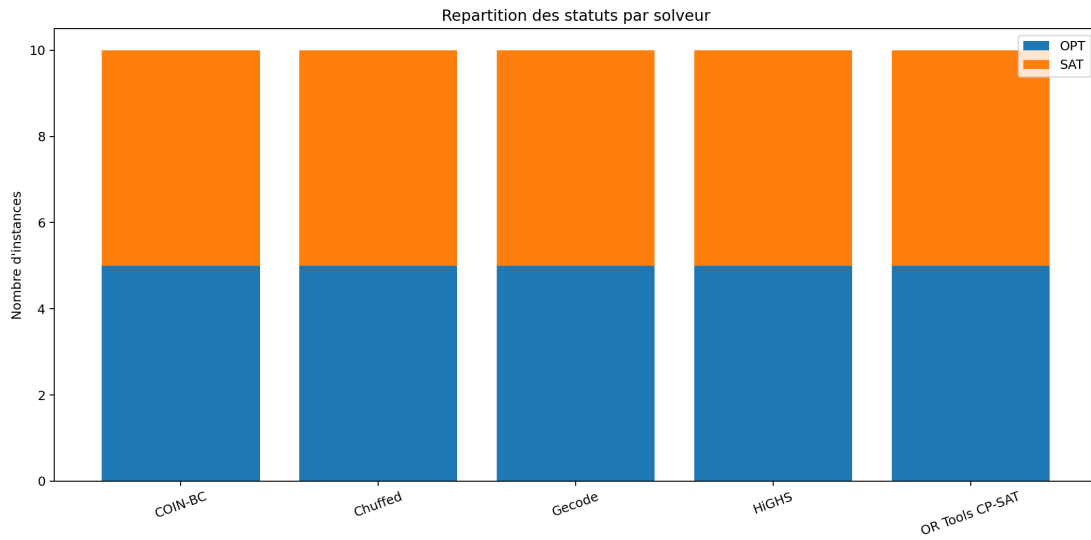


FIGURE 6 – Répartition des statuts retournés par solveur

La figure 7 enfonce le même résultat sous l’angle des taux agrégés : 100% de succès et 0% de timeout pour tous les solveurs. Autrement dit, la robustesse ne permet pas ici de départager les candidats. Ce sont la vitesse et surtout l’évolution de cette vitesse avec la taille qui deviennent les critères discriminants.

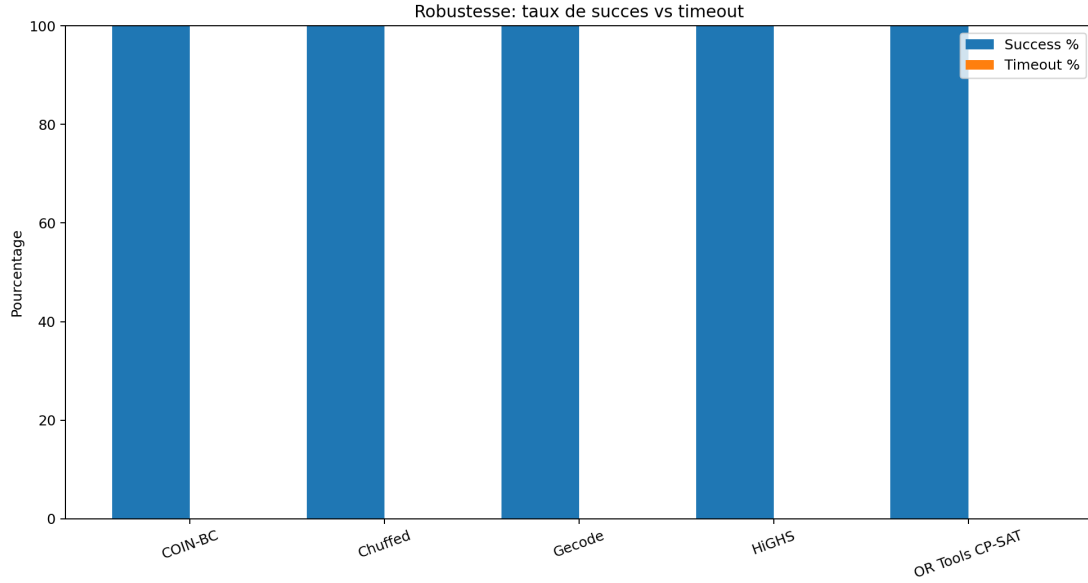


FIGURE 7 – Taux de succès et de timeout observés pendant la campagne

La figure 8 propose enfin une lecture cumulative. Le cactus plot ordonne les instances par difficulté croissante et cumule les temps de résolution. Une courbe plus basse signifie que le solveur accumule plus vite les instances résolues. CHUFFED, GECODE et OR-TOOLS CP-SAT dominent nettement la partie basse du graphe, tandis que HIGHS et COIN-BC progressent plus lentement. Cette représentation est précieuse car elle évite qu’une seule valeur moyenne masque la dynamique globale d’un solveur sur tout le benchmark.

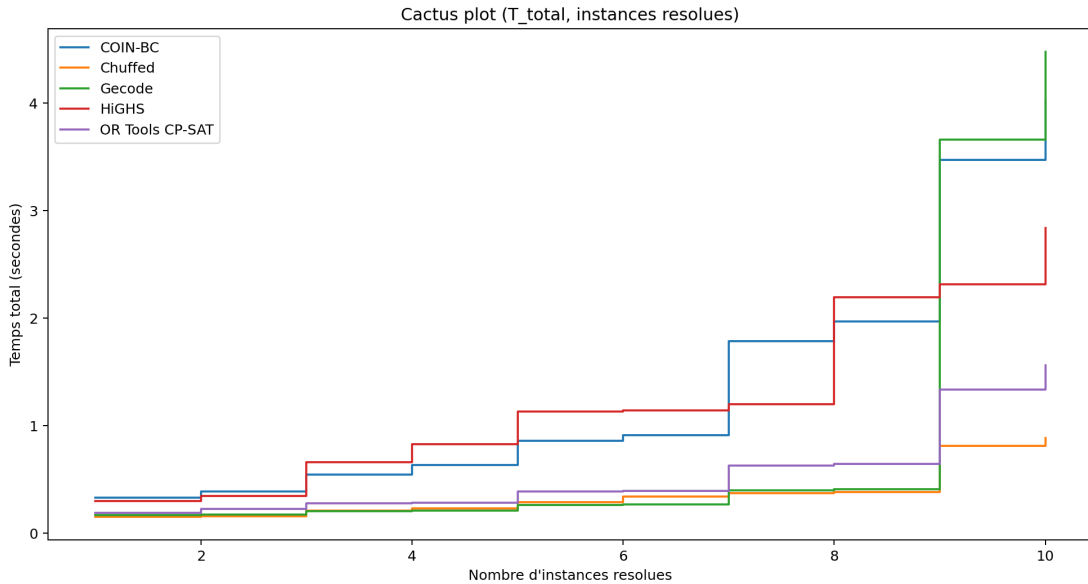


FIGURE 8 – Cactus plot basé sur le temps total de résolution

10 Analyse du passage à l’échelle

Les courbes par taille sont essentielles, car le but du mémoire n’est pas seulement de résoudre de petites instances, mais d’identifier le solveur qui reste pertinent quand

le nombre d'activités augmente. La taille A sert ici de paramètre directeur, avec une croissance synchronisée du nombre d'enseignants, de salles et de créneaux.

La figure 9 montre que GECODE conserve un léger avantage sur les tailles intermédiaires, notamment pour $A = 4$ et $A = 5$. Dès que l'on atteint $A = 6$, puis surtout $A = 8$, CHUFFED devient nettement plus favorable. À $A = 8$, la hiérarchie change clairement : environ 0.849 seconde pour CHUFFED, 1.449 seconde pour OR-TOOLS CP-SAT, 2.578 secondes pour HIGHS, 3.746 secondes pour COIN-BC et 4.067 secondes pour GECODE. Ce point justifie à lui seul qu'on ne retienne pas uniquement la médiane globale.

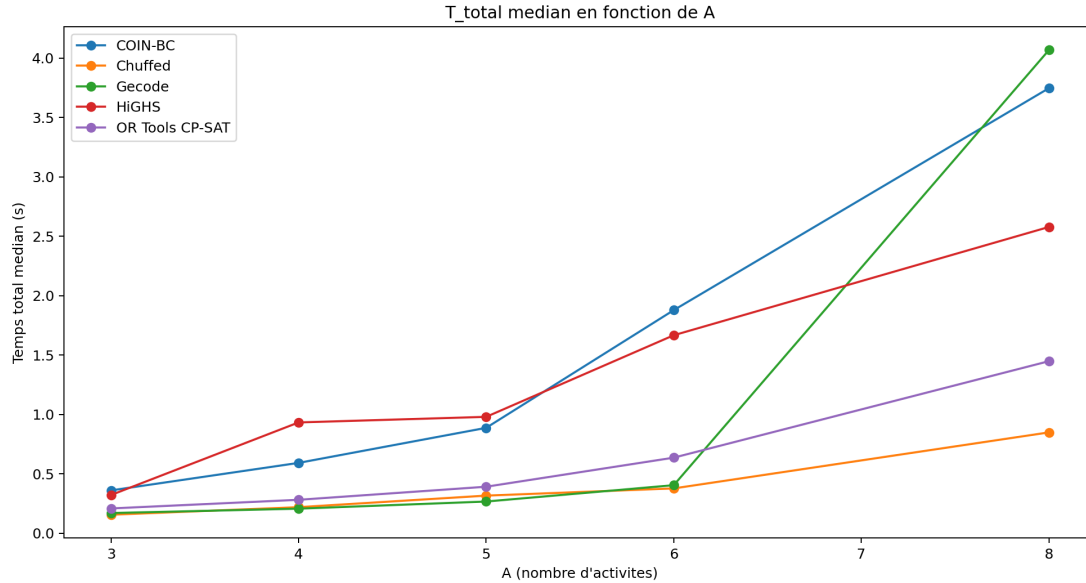


FIGURE 9 – Évolution du temps total médian en fonction du nombre d'activités A

La figure 10 confirme cette tendance lorsque l'on observe uniquement le temps solveur. CHUFFED devient alors le plus compétitif sur la partie haute de la gamme. Cette observation s'accorde avec la littérature sur la lazy clause generation : l'apprentissage de clauses exploite les conflits rencontrés pendant la recherche et aide à éviter des explorations redondantes [5].

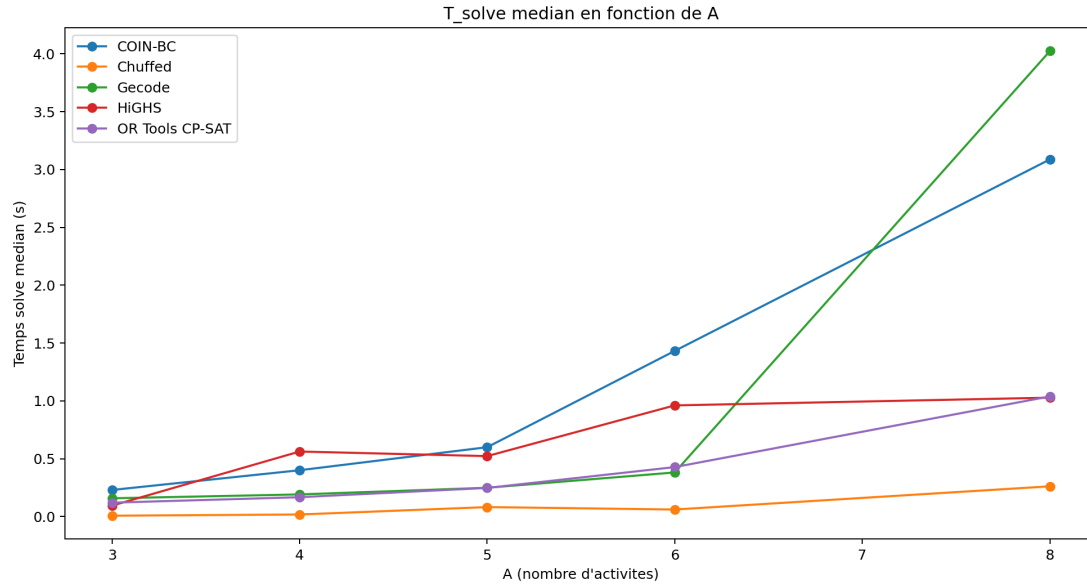


FIGURE 10 – Évolution du temps de recherche médian en fonction du nombre d'activités A

La figure 11 complète cette analyse en montrant que la montée en taille, dans la plage étudiée, ne dégrade pas encore le taux de succès. C'est un point positif pour le protocole : les instances restent suffisamment maîtrisées pour produire une comparaison nette. En revanche, cela signifie aussi qu'une future campagne devra pousser plus loin la difficulté si l'on veut différencier les solveurs sur la robustesse et non plus seulement sur le temps.

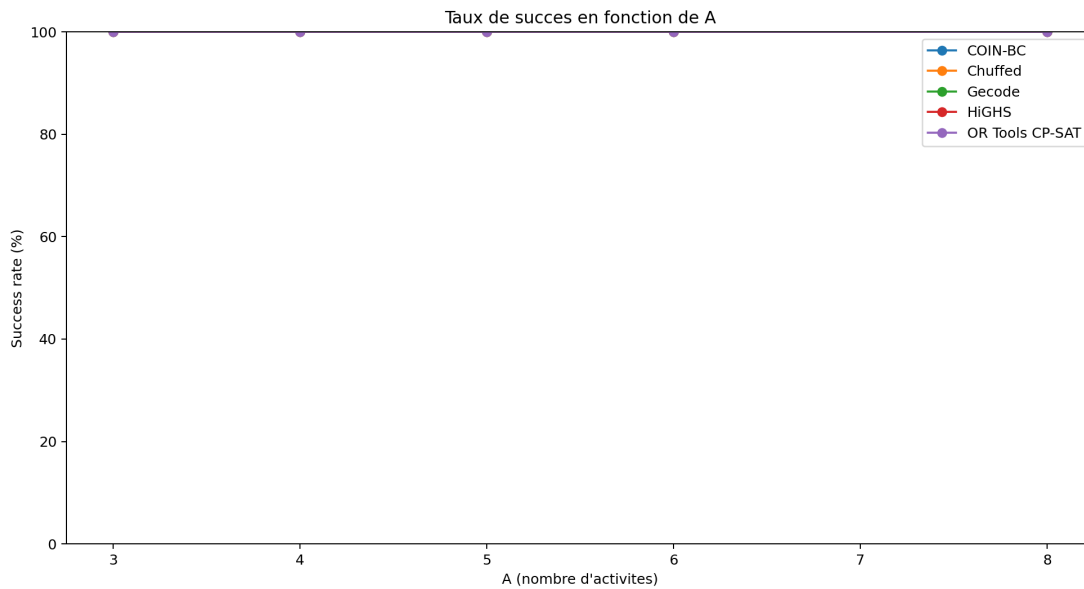


FIGURE 11 – Taux de succès en fonction du nombre d'activités A

11 Interprétation par la littérature

11.1 GECODE : CP classique et petites instances

GECODE est un solveur de programmation par contraintes sur domaines finis. Il est conçu pour exploiter la propagation de contraintes et dispose d'un support riche de contraintes globales dans l'écosystème MiniZinc [1, 3]. Sur les petites instances de cette campagne, il obtient la meilleure médiane globale et la meilleure médiane dans les deux familles. Ce résultat est cohérent : les instances sont simples, fortement contraintes par des affectations fixées, et ne nécessitent pas une exploration très profonde.

Cependant, le passage à $A = 8$ montre une dégradation plus forte. Cela ne signifie pas que GECODE est mauvais ; cela indique seulement que, dans notre formulation actuelle, son avantage initial peut diminuer lorsque la combinatoire des affectations augmente.

11.2 CHUFFED : lazy clause generation et montée en taille

CHUFFED utilise la lazy clause generation, une approche hybride combinant propagation CP et apprentissage de clauses [5]. Cette stratégie est pertinente pour des modèles comportant de nombreuses décisions booléennes, des affectations et des conflits de ressources. Dans notre benchmark, CHUFFED obtient la meilleure moyenne globale, le meilleur temps solveur médian et le meilleur comportement sur les plus grandes instances.

Cette observation est importante pour le mémoire : même si GECODE gagne à la médiane sur des instances simples, CHUFFED est le solveur qui semble le mieux préparer le passage à des instances plus grandes ou plus combinatoires.

11.3 OR-TOOLS CP-SAT : candidat hybride solide

OR-TOOLS CP-SAT est un solveur hybride pour problèmes entiers et booléens. Sa documentation insiste sur les variables entières, les contraintes discrètes et les statuts de faisabilité ou d'optimalité [6]. Dans cette campagne, il n'est jamais premier, mais il reste régulier et arrive souvent après CHUFFED et GECODE. Ce comportement en fait un candidat stratégique à conserver dans les campagnes futures, notamment lorsque les préférences ou les contraintes logiques deviendront plus nombreuses.

11.4 HIGHS et COIN-BC : solveurs MILP

HIGHS et COIN-BC appartiennent à la famille des solveurs d'optimisation linéaire en nombres entiers. HIGHS vise les problèmes LP, MIP et QP [7, 8] ; COIN-BC est un solveur branch-and-cut open source [9]. Ils sont donc naturellement pertinents lorsque le modèle MiniZinc se traduit en une formulation MILP forte.

Dans cette campagne, ils résolvent toutes les instances et prouvent l'optimalité sur les cas d'optimisation, mais ils sont plus lents que les solveurs CP/LCG. Cela suggère que notre formulation actuelle reste d'abord combinatoire : décisions d'affectation, rôles, ressources et non-chevauchement. Les relaxations linéaires ne semblent pas apporter, sur ces instances, un avantage suffisant pour compenser le coût de la traduction MILP.

12 Décision expérimentale

La conclusion doit distinguer trois niveaux.

1. **Petites instances simples.** GECODE est le meilleur choix si l'on retient strictement la médiane globale de `T_total`. Il est très efficace sur les instances centrales du benchmark.
2. **Passage à l'échelle.** CHUFFED est le meilleur candidat lorsque A augmente. Il obtient la meilleure moyenne globale et le meilleur comportement sur la plus grande taille testée.
3. **Solveur par défaut pour la plateforme.** Pour une plateforme destinée à évoluer vers des instances plus grandes et plus contraintes, CHUFFED est le choix le plus prudent comme solveur par défaut, avec GECODE comme alternative rapide sur petites instances de satisfaction.

La recommandation finale est donc :

CHUFFED doit être retenu comme solveur par défaut expérimental pour la plateforme, car il combine un très bon temps de recherche, une robustesse complète et une meilleure tenue lorsque la taille augmente. GECODE doit rester disponible comme alternative très performante sur les petites instances simples.

13 Limites

La campagne présentée dans ce chapitre correspond à une première phase volontairement simple, construite pour garantir des résolutions rapides et une lecture propre des résultats. Ce choix a permis de comparer les solveurs sans bruit expérimental, mais il limite encore la portée des conclusions. Les résultats n'épuisent pas le comportement des solveurs sur des tailles plus hautes ni sur des variantes un peu plus tendues.

Les limites principales sont :

- une seule seed par taille ;
- seulement cinq tailles ;
- des préférences volontairement légères ;
- absence d'instances infaisables ;
- absence de variantes très contraintes en fenêtres temporelles ou disponibilités ;
- pas encore d'étude détaillée des options de recherche propres à chaque solveur.

Pour pallier ces limites, l'outillage expérimental a été étendu dans une seconde configuration de benchmark plafonnée à 300 secondes par couple solveur-instance. Cette extension conserve les deux familles **satisfaction** et **optimisation**, mais ajoute trois tailles supplémentaires, dont deux nouvelles tailles à $A = 15$ et $A = 20$. Elle introduit aussi une difficulté graduelle à l'intérieur de chaque famille : légère baisse du taux de jury fixé, quelques précédences, quelques fenêtres temporelles, quelques interdictions d'affectation et, pour l'optimisation, un ensemble de préférences un peu plus riche à partir des grandes tailles.

La famille ne doit donc pas être confondue avec la difficulté. Une instance de satisfaction peut rester simple ou devenir plus structurée ; une instance d'optimisation peut

conserver un objectif léger ou intégrer davantage de préférences. Cette séparation est essentielle pour la suite du mémoire, car elle permet d’analyser indépendamment la nature du problème et son niveau réel de complexité.

14 Conclusion

Cette campagne apporte une contribution concrète au mémoire : elle établit une méthode reproductible pour comparer les solveurs de la plateforme DSL-MiniZinc. Les instances sont identifiées par famille, taille et seed ; les métriques séparent compilation, recherche et temps total ; les résultats sont interprétés par famille, par statut et par taille.

Les résultats montrent que tous les solveurs testés sont capables de résoudre les instances actuelles. La différence ne se situe donc pas dans la correction, mais dans l’efficacité. GECODE est le plus rapide en médiane sur ces petites instances. CHUFFED est le plus intéressant pour la plateforme parce qu’il domine en moyenne, en temps solveur et en passage à l’échelle. OR-TOOLS CP-SAT reste un candidat solide. HiGHS et COIN-BC sont utiles comme références MILP, mais ne sont pas les meilleurs choix pour la formulation actuelle.

La conclusion scientifique est donc nuancée et défendable : le choix du solveur dépend de la famille, de la taille et de la formulation générée. Pour l’état actuel du DSL, CHUFFED apparaît comme le meilleur solveur par défaut, tandis que GECODE constitue une excellente alternative pour les instances simples de satisfaction.

Références

- [1] MiniZinc Handbook, *Solving Technologies and Solver Backends*, <https://docs.minizinc.dev/en/latest/solvers.html>.
- [2] MiniZinc, *MiniZinc Challenge*, <https://www.minizinc.org/challenge/>.
- [3] Gecode, *An open, free, efficient constraint solving toolkit*, <https://www.gecode.dev/>.
- [4] Chuffed, *The Chuffed CP solver*, <https://github.com/chuffed/chuffed>.
- [5] T. Feydy and P. J. Stuckey, *Lazy clause generation reengineered*, Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming, 2009.
- [6] Google OR-Tools, *CP-SAT Solver*, https://developers.google.com/optimization/cp/cp_solver.
- [7] HiGHS, *High-performance parallel linear optimization software*, <https://highs.dev/>.
- [8] Q. Huangfu and J. A. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation, 10(1), 119–142, 2018. <https://doi.org/10.1007/s12532-017-0130-5>.
- [9] COIN-OR, *Introduction to CBC*, <https://coin-or.github.io/Cbc/intro.html>.